SMART FINANCE TRACKER

Version 2.0 • Technical Master Document & Code Manual

Author:	Nachiket K.
Status:	Completed, Verified, & Deployed
Environment:	Python 3.10+ & Streamlit Local/Cloud
Database:	SQLite (Local & Offline)
Date:	June 23, 2026

1. Executive Overview & Architecture

The **Smart Finance Tracker v2.0** is a production-quality, offline-first personal finance application built with Python, Streamlit, Pandas, Plotly, and SQLite. The design philosophy strictly prioritizes **user privacy and data security** by running all computational layers, analytical databases, and artificial intelligence engines locally. No data leaves the user's host machine, eliminating the need for cloud dependencies, paid API subscription tokens, or third-party tracking.

The application architecture consists of a layered framework separating database CRUD structures, math and ML aggregations, rule-based reasoning for financial alerts, multi-format file parsers, and the Streamlit UI presentation layer. This enforces single-responsibility patterns across files and prevents codebase spaghetti.

Directory Component Layout:

File	Role in System Architecture
app.py	Main Streamlit entry point. Manages session states, pages routing, sidebar navigation, HTML/CSS loading, form
database.py	Data Access Object (DAO) layer. Creates the SQLite schema, handles dynamic column migrations, reset scripts
auth.py	User authentication service. Handles registration, password hashing (bcrypt), login queries, and secure session
analytics.py	Mathematical engine. Groups pandas transactions, fits Linear Regression prediction models, structures Plotly gr
ai_insights.py	Rule-based analysis engine. Scans transaction sheets for unusual spending anomalies, recurring subscriptions,
bank_parser.py	Normalized Bank Statement importer. Maps columns and checks for duplicate transactions prior to SQLite datab
ocr_parser.py	UPI receipt screen reader. Executes local Tesseract OCR engine scans to pull amounts, dates, and merchant na
gmail_parser.py	Gmail alert sync manager. Authenticates desktop OAuth scopes to read-only scan bank alert subject lines and a
export_utils.py	Data output compiler. Formats workbook xlsxwriter sheets and prints vector-drawn ReportLab PDF files containi
utils.py	Utility helpers. Holds static maps, color hex constants, currency string formatters, regex string cleaners, and san

2. Database Schema Design & Dynamic Migrations

Smart Finance Tracker utilizes a local SQLite database (`finance.db`) stored under `finance_tracker/data/`. The relational database schema is structured around six tables: `users`, `settings`, `transactions`, `budgets`, `goals`, and `recurring_transactions`.

SQLite Schema Definitions:

users: Tracks user credentials and registration timestamps. Password hashes are stored as bytes.
settings: Stores user settings like selected currency (e.g. INR, USD) and theme choice (e.g. dark, light).
transactions: Primary financial entry table. Records transaction type, amount, category, notes, and transaction date.
budgets: Sets monthly limits per expense category for a specific month string (YYYY-MM).
goals: Stores savings goal names, targets, saved amounts, deadlines, and priorities.
recurring_transactions: Stores template records for weekly/monthly recurring payments (e.g., subscriptions, salaries).

Dynamic Schema Migration Logic Walkthrough:

To support backward compatibility and upgrade older v1.0 databases to v2.0, an inline migration block is executed dynamically during `init_db()` in [database.py](file:///c:/Users/nachi/OneDrive/Desktop/CS-G%20Batch-1%20Group%201%20Python%20course%20project-personal%20expanse%20tracker/finance_tracker/database.py). Rather than relying on heavy migration frameworks (like Alembic), the application uses SQLite's built-in `PRAGMA table_info` columns checking. It queries existing column names and alters tables dynamically if needed:

```
def init_db():
    conn = get_connection()
    c = conn.cursor()
    # (Standard table creation omitted for brevity...)

    # ■■ Migrations / Alter columns if they don't exist ■■
    # Check transactions columns
    c.execute("PRAGMA table_info(transactions)")
    txn_cols = [row[1] for row in c.fetchall()]
    if "tags" not in txn_cols:
        c.execute("ALTER TABLE transactions ADD COLUMN tags TEXT")

    # Check goals columns
    c.execute("PRAGMA table_info(goals)")
    goal_cols = [row[1] for row in c.fetchall()]
    if "priority" not in goal_cols:
        c.execute("ALTER TABLE goals ADD COLUMN priority TEXT DEFAULT 'Medium'")

    conn.commit()
    conn.close()
```

JSON Profile Backup & Restore Mechanics:

To allow users to export and migrate their entire profiles between local machines, a secure JSON backup system was built. `backup_user_data(user_id)` builds a consolidated Python dictionary of all rows mapped to that `user_id` across all five relational data tables, and returns it as a portable JSON object.

The restore function `restore_user_data(user_id, data)` does the reverse. It executes inside a database transaction block. It wipes all existing user records, iterates through the tables in the backup dictionary, inserts the rows securely mapped to the current active `user_id`, and runs a `conn.commit()`. If any error

occurs (e.g., malformed JSON data, broken constraints), it automatically triggers `conn.rollback()` to prevent corruption.

3. Authentication & Password Security

The security subsystem in [auth.py](file:///c:/Users/nachi/OneDrive/Desktop/CS-G%20Batch-1%20Group%201%20Python%20course%20project-personal%20expanse%20tracker/finance_tracker/auth.py) implements secure registration and login routines. User passwords are hashed using **bcrypt**, a cryptographically secure hash function based on the Blowfish cipher. Bcrypt automatically handles salt generation locally, protecting hashes from precomputed rainbow table attacks.

Hashing Technique: When a user registers, `bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())` generates a salted hash which is stored in the `password` field of the `users` SQLite table. During login verification, `bcrypt.checkpw(password.encode('utf-8'), hashed_password_from_db)` evaluates the match in constant time, defending against timing attacks.

Session State Handling: Authentication state is persisted locally inside Streamlit's `st.session_state` dictionary using key indicators: `st.session_state['logged_in']` (Boolean), `st.session_state['user_id']` (Integer), and `st.session_state['username']` (String). Upon logout, these keys are cleared and the app triggers `st.rerun()` to force redraw the login page, blocking unauthorized access.

Authentication Helper Methods:

```
def login(username, password):
    user = get_user_by_username(username)
    if not user:
        return False, "■ Invalid credentials"
    # Check hashed password matches
    if bcrypt.checkpw(password.encode('utf-8'), user['password']):
        st.session_state["logged_in"] = True
        st.session_state["user_id"] = user["id"]
        st.session_state["username"] = user["username"]
        return True, "■ Login successful!"
    return False, "■ Invalid credentials"
```

4. Analytics, Extrapolation, & Caching Engine

The analytics subsystem in [analytics.py](file:///c:/Users/nachi/OneDrive/Desktop/CS-G%20Batch-1%20Group%201%20Python%20course%20project-personal%20expanse%20tracker/finance_tracker/analytics.py) computes key performance metrics and trends and compiles interactive Plotly visualisations.

Financial Health Score (0-100) Algorithm: Evaluates a user's financial habits based on three key metrics: 1. *Savings Rate (45% weight)*: Full points (45) if savings rate is >=30%; scales down linearly to 0 if negative. 2. *Budget Discipline (35% weight)*: Deducts points based on the fraction of active categories exceeding their monthly budgets. 3. *Savings Goals Progress (20% weight)*: Evaluates aggregate progress (saved amount vs target) across all active goals.

Spending Trend Prediction (scikit-learn Linear Regression): Fits a linear model on cumulative daily expenses. Given daily spending sequences $(X = \text{day of month}, Y = \text{cumulative spend})$, the model fits a regression line $Y = mX + c$ using Ordinary Least Squares (OLS). It extrapolates the daily coefficients to forecast total month-end spending and visualizes the forecast as a dashed regression path alongside the actual solid spending path.

Dynamic Caching & Invalidation: To keep the UI fast and snappy, slow database queries are cached using `@st.cache_data`. However, to prevent stale cache data, we generate a database summary trigger key:

`f"{row_count}_{max_id}_{sum_amount}"`. This key is passed into the cached function parameters. Whenever a transaction is inserted, updated, or deleted, the sum or count changes, modifying the trigger key and automatically invalidating the cache without manual refreshes.

5. AI Insights Engine (100% Local)

The artificial intelligence layer in [ai_insights.py](file:///c:/Users/nachi/OneDrive/Desktop/CS-G%20Batch-1%20Group%201%20Python%20course%20project-personal%20expanse%20tracker/finance_tracker/ai_insights.py) executes a heuristics-based evaluation engine to analyze cash flows, budgets, and savings rates offline. Key analytical rules include:

• **Anomalous Spike Detection**: Identifies individual transactions exceeding $2.5\times$ the user's overall mean transaction size, flagging them as potential budget leaks.
• **Emergency Fund Reserves**: Audits savings totals and alerts if the reserve fund is below the recommended three months of average expenses.
• **Subscription Cost Aggregation**: Extracts recurring keywords from descriptions (e.g. Netflix, Spotify) to aggregate and alert on recurring monthly subscription costs.
• **Weekend vs. Weekday Ratio**: Grouping transactions by $(X = \text{day of week})$, the system flags if weekend spending significantly outpaces weekday habits.

■ ***Local AI Philosophy:*** *All insights are computed on-device using local Pandas aggregations and scikit-learn models. This guarantees absolute data privacy and keeps the application fast and free.*

6. Import & Parsing Pipeline (OCR, Gmail, Deduplication)

The import module handles bank statements, screenshot OCR parsing, Gmail alert synchronization, and duplicate detection.

Universal Bank Statement Parser: Parses net banking CSV, Excel, and PDF files. Using preconfigured columns in `BANK_PROFILES` for 14+ Indian banks (such as SBI, HDFC, ICICI, Axis, Kotak), it automatically detects the bank format, extracts dates, descriptions, debits, credits, and reference numbers, and categorizes transactions using local keyword maps.

Duplicate Pre-Check Flagging: In `flag_duplicate_transactions_in_df()`, the uploaded statement is cross-referenced against the SQLite database. A unique key tuple `(date, amount, description[:30])` is generated for each row. Transactions are flagged as 'New', 'Duplicate (Exists)' (matching database records), or 'Duplicate (File)' (repeating inside the file). The preview table displays these flags, and duplicate entries are skipped during the import run to prevent data double-counting.

UPI Screenshot OCR via Tesseract: Extracts text from GPay, PhonePe, and Paytm screenshots. It scans the raw image bytes using `pytesseract.image_to_string()`, extracts transaction metadata (amounts, dates, merchants, reference numbers) using regex, and presents them in an editable form for quick insertion.

Gmail Alerts Synchronization: Scans the user's mailbox for bank alerts from banks like HDFC, SBI, and ICICI. It connects via standard OAuth client flow, retrieves messages from the last N days, extracts payment amounts and merchants, and auto-records transactions locally.

7. User Interface & General Code Quality

The user interface in [app.py](file:///c:/Users/nachi/OneDrive/Desktop/CS-G Batch-1 Group 1 Python course project-personal expanse tracker/finance_tracker/app.py) uses a responsive grid layout styled via [style.css](file:///c:/Users/nachi/OneDrive/Desktop/CS-G Batch-1 Group 1 Python course project-personal expanse tracker/finance_tracker/assets/style.css). Key interface improvements include:

• **Dashboard Growth deltas**: Streamlit metric columns show MoM growth percentages dynamically.
• **Transactions Edit & Duplication**: Interactive forms with inline selectors allow editing and cloning transactions easily.
• **Budget Warn Bars**: Color-coded progress bars (green $\le$ 75%, yellow $\le$ 100%, red over budget) and overspending warning banners flag budget issues early.
• **Goal Progress & Track Flags**: Displays days remaining and expected completion dates, showing On/Off Track flags based on target deadlines and savings rates.

Deployment Path-Fix Analysis:

When running local Streamlit apps, the parent folder is typically added to `sys.path`. However, on cloud deployment servers (like Streamlit Cloud), the root directory is `/mount/src/smart-finance-tracker/` and Python cannot resolve subfolder files. To prevent `ImportError` crashes, we added a path resolution hook at the top of [app.py](file:///c:/Users/nachi/OneDrive/Desktop/CS-G Batch-1 Group 1 Python course project-personal expanse tracker/finance_tracker/app.py):

```
import sys
import os
# Add current directory to path for local module imports in deployed environments
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
```

8. Detailed File Walkthrough

• app.py

Acts as the application entry point. Handles rendering for all page tabs: Dashboard (KPI cards, charts, recent activities), Transactions (add/edit/duplicate forms, tags filter, bulk delete), Import (Tesseract OCR upload, statement reader, OAuth synchronization), Analytics, Budget, Goals (detail forms, predictions), Export, and Settings.

• database.py

Encapsulates database operations. Configures schemas, check migrations, and handles CRUD operations for user profiles, transaction tags, budgets, and goals, as well as backup/restore exports.

• auth.py

Handles authentication logic. Salts and hashes passwords using bcrypt, queries username existence, checks credentials, and manages secure session states.

• analytics.py

Aggregates transaction data into Pandas DataFrames. Builds Plotly charts, computes financial health scores, and trains scikit-learn models to predict monthly spending.

• ai_insights.py

Evaluates spending patterns locally. Flags anomalies, estimates reserve funds, aggregates subscriptions, and generates monthly performance summaries.

• bank_parser.py

Parses CSV, Excel, and PDF bank statements. Normalizes columns, parses dates, categorizes transactions, and flags duplicate transactions beforehand.

• ocr_parser.py

Scans payment receipts and transaction screenshots locally using Tesseract OCR, extracting details with regex.

• gmail_parser.py

Syncs transactions via Gmail. Utilizes Google OAuth flow, retrieves message content, parses transaction alerts, and updates local records.

• export_utils.py

Formats and exports data. Generates Excel workbooks with styled sheets and creates professional ReportLab PDF summary reports.

• utils.py

Stores constants, categories, currency formats, and inputs validation rules, and loads mock demo data.

9. Verification & Conclusion

All updates in the **Smart Finance Tracker v2.0** release are verified and fully functional. The codebase adheres to clean programming principles, single-responsibility logic, and secure password-handling rules.

Verification Steps Completed:

1. Dynamic database column migrations successfully checked on startup.
2. Streamlit Cloud path-resolution hooks tested to prevent import crashes.
3. Local AI Insight models and regression prediction curves verified.
4. PDF export layouts and Excel sheets tested successfully.

Document End • Smart Finance Tracker v2.0 • Local & Private Finance Companion